

**Aim:** Merge sort

**Theory:** Merge sort is defined as a sorting algorithm that works by dividing an array into smaller subarrays, sorting each subarray, and then merging the sorted subarrays back together to form the final sorted array.

**Time Complexity:**

- Best Case:  $O(n \log n)$
- Worst Case:  $O(n \log n)$
- Average Case:  $O(n \log n)$

The time complexity of Merge Sort is consistently  $O(n \log n)$  regardless of the input arrangement, making it a reliable and efficient sorting algorithm.

**Algorithm:**

- MergeSort(A, p, r)
  1. If  $p < r$
  2.  $q = \lfloor \frac{p+r}{2} \rfloor$
  3. MergeSort(A, p, q)
  4. MergeSort(A, q+1, r)
  5. Merge(A, p, q, r)

```
MERGE(A, p, q, r)
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1 \dots n_1 + 1]$  and  $R[1 \dots n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```

**Aim:** Quick sort

**Theory:** QuickSort is a sorting algorithm based on the Divide and Conquer algorithm that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.

### Time Complexity

- Best case:  $O(n \log n)$
- Average Case:  $O(n \log n)$
- Worst Case:  $O(n^2)$

### Algorithm

```
quicksort (array){  
    if (array.length > 1){  
        choose a pivot;  
        while (there are items left in array){  
            if (item < pivot)  
                put item into subarray1;  
            else  
                put item into subarray2;  
        }  
        quicksort(subarray1);  
        quicksort(subarray2);  
    }  
}
```

**Aim:** Bubble Sort

**Theory:** Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity is quite high.

## Algorithm

```
Algorithm Bubblesort(input: n, V)
def n : number of values to sort;
def V[n] : array of size n;
def temp, i, j: integer variables;
1. i := n - 1;
2. while ( i >= 1 ) do
3.     j := 0;
4.     while ( j < i ) do
5.         if ( V[j] < V[j+1] )
6.             temp := V[j];
7.             V[j] := V[j+1];
8.             V[j+1] := temp;
9.         end if
10.        j := j + 1;
11.    end while
12.    i := i - 1;
13. end while
14. return V;
```

## Time Complexity

- Worst Case:  $O(n^2)$
- Average Case:  $O(n^2)$
- Best Case:  $O(n)$

Bubble Sort has a quadratic time complexity in the worst and average cases and linear time complexity in the best case. It's not considered efficient for large datasets, and other sorting algorithms are often preferred.

**Aim:** Selection Sort

**Theory:** Selection sort is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the list and moving it to the sorted portion of the list.

## Algorithm

### Selection Sort – Pseudocode

**Input:** An array  $A[1..n]$  of  $n$  elements.

**Output:**  $A[1..n]$  sorted in descending order

1. for  $i \leftarrow 1$  to  $n - 1$
2.    $\min \leftarrow i$
3.   for  $j \leftarrow i + 1$  to  $n$  {Find the  $i$ th smallest element.}
4.     if  $A[j] < A[\min]$  then
5.        $\min \leftarrow j$
6.   end for
7.   if  $\min \neq i$  then interchange  $A[i]$  and  $A[\min]$
8. end for

22

## Time Complexity

- Best-case:  $O(n^2)$ , best case occurs when the array is already sorted.  
(where  $n$  is the number of integers in an array)
- Average-case:  $O(n^2)$ , the average case arises when the elements of the array are in a disordered or random order, without a clear ascending or descending pattern.

Worst-case:  $O(n^2)$ , The worst-case scenario arises when we need to sort an array in ascending order, but the array is initially in descending order.

**Aim:** Heap Sort

**Theory:** Heap sort is a comparison-based sorting technique based on Binary Heap data structure. It is similar to the selection sort where we first find the minimum element and place the minimum element at the beginning. Repeat the same process for the remaining elements.

## Algorithm

*For Problems #4 through #6, consider the following pseudocode (ref: CLRS pp.154–160):*

```
01 HEAPSORT (A)
02   BUILD-MAX-HEAP (A)
03   for i = A.length downto 2
04     exchange A[1] and A[i]
05     A.heapSize = A.heapSize - 1
06     MAX-HEAPIFY (A,1)

07 BUILD-MAX-HEAP (A)
08   A.heapSize = A.length
09   for i = floor(A.length/2) downto 1
10     MAX-HEAPIFY (A,i)

11 MAX-HEAPIFY (A,i)
12   L = 2*i
13   R = 2*i + 1
14   if L <= A.heapSize and A[L] > A[i]
15     largest = L
16   else
17     largest = i
18   if R <= A.heapSize and A[R] > A[largest]
19     largest = R
20   if largest != i
21     exchange A[i] and A[largest]
22     MAX-HEAPIFY (A,largest)
```

## Time Complexity

- Best Case:  $O(n \log n)$
- Worst Case:  $O(n \log n)$
- Average Case:  $O(n \log n)$

**Aim:** Linear Search

**Theory:** Linear Search is defined as a sequential search algorithm that starts at one end and goes through each element of a list until the desired element is found, otherwise the search continues till the end of the data set.

### Algorithm

**Algorithm**

Step 1: Start  
Step 2: Declare array a and variables flag, i, n  
Step 3: input for array and initialize flag <- 0  
Step 4: input for n, the number to be searched  
Step 5: if (a[i] = n)  
    flag = 0  
    then  
        Display Number is present at location (i+1)  
Step 6: if (flag = 1)  
    then  
        Display Number is not present in the array  
Step 7: Stop

www.geekyshows.com

### Time Complexity

- Best Case: In the best case, the key might be present at the first index. So the best case complexity is  $O(1)$
- Worst Case: In the worst case, the key might be present at the last index i.e., opposite to the end from which the search has started in the list. So the worst-case complexity is  $O(N)$  where  $N$  is the size of the list.

Average Case:  $O(N)$

**Aim:** Binary Search

**Theory:** Binary search is a search algorithm that efficiently finds the position of a target value within a sorted array. The key feature of binary search is that it repeatedly divides the search interval in half, drastically reducing the search space with each comparison.

### Time Complexity < Binary Search >

- Worst-case time complexity:  $O(\log n)$
- Average-case time complexity:  $O(\log n)$
- Best-case time complexity:  $O(1)$  (when the target is at the middle position)

### Algorithm

```
procedure bubbleSort(A : list of sortable items)
  n = length(A)
  repeat
    newn = 0
    for i = 1 to n-1 inclusive do
      if A[i-1] > A[i] then
        swap(A[i-1], A[i])
        newn = i
      end if
    end for
    n = newn
  until n = 0
end procedure
```

**Aim:** Strassen's matrix multiplication

**Theory:** Strassen's Matrix Multiplication is the divide and conquer approach to solve the matrix multiplication problems. The usual matrix multiplication method multiplies each row with each column to achieve the product matrix. The time complexity taken by this approach is  $O(n^3)$ , since it takes two loops to multiply. Strassen's method was introduced to reduce the time complexity from  $O(n^3)$  to  $O(n \log 7)$ .

### Time Complexity

$$T(n) = \begin{cases} c & \text{if } n = 1 \\ 7 \times T\left(\frac{n}{2}\right) + d \times n^2 & \text{otherwise} \end{cases} \text{ where } c \text{ and } d \text{ are constants}$$

Using this recurrence relation, we get  $T(n) = O(n^{\log 7})$

Hence, the complexity of Strassen's matrix multiplication algorithm is  $O(n^{\log 7})$ .

### Algorithm



$$\begin{aligned}
p1 &= a(f - h) & p2 &= (a + b)h \\
p3 &= (c + d)e & p4 &= d(g - e) \\
p5 &= (a + d)(e + h) & p6 &= (b - d)(g + h) \\
p7 &= (a - c)(e + f)
\end{aligned}$$

The A x B can be calculated using above seven multiplications.

Following are values of four sub-matrices of result C

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} p5 + p4 - p2 + p6 & p1 + p2 \\ p3 + p4 & p1 + p5 - p3 - p7 \end{bmatrix}$$

A                      B                      C

A, B and C are square matrices of size N x N

a, b, c and d are submatrices of A, of size N/2 x N/2

e, f, g and h are submatrices of B, of size N/2 x N/2

p1, p2, p3, p4, p5, p6 and p7 are submatrices of size N/2 x N/2

### Algorithm 3 Strassen's Matrix Multiplication Algorithm

**Input:**  $A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$  and  $B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \in \mathbb{R}^{n \times n}$

- 1: **if**  $n = 1$  **then**
- 2:      $C = A \cdot B$
- 3: **else**
- 4:      $M_1 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$
- 5:      $M_2 = (A_{21} + A_{22}) \cdot B_{11}$
- 6:      $M_3 = A_{11} \cdot (B_{12} - B_{22})$
- 7:      $M_4 = A_{22} \cdot (B_{21} - B_{11})$
- 8:      $M_5 = (A_{11} + A_{12}) \cdot B_{22}$
- 9:      $M_6 = (A_{21} - A_{11}) \cdot (B_{11} + B_{12})$
- 10:     $M_7 = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$
- 11:     $C_{11} = M_1 + M_4 - M_5 + M_7$
- 12:     $C_{12} = M_3 + M_5$
- 13:     $C_{21} = M_2 + M_4$
- 14:     $C_{22} = M_1 - M_2 + M_3 + M_6$

**Output:**  $A \cdot B = C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} \in \mathbb{R}^{n \times n}$

In this context, using Strassen's Matrix multiplication algorithm, the time consumption can be improved a little bit.

Strassen's Matrix multiplication can be performed only on square matrices where  $n$  is a power of 2. Order of both of the matrices are  $n \times n$ .

Divide  $X$ ,  $Y$  and  $Z$  into four  $(n/2) \times (n/2)$  matrices as represented below –

$$Z = \begin{bmatrix} I & J \\ K & L \end{bmatrix} \quad X = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \quad \text{and} \quad Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix}$$

Using Strassen's Algorithm compute the following –

$$M_1 := (A + C) \times (E + F)$$

$$M_2 := (B + D) \times (G + H)$$

$$M_3 := (A - D) \times (E + H)$$

$$M_4 := A \times (F - H)$$

$$M_5 := (C + D) \times (E)$$

$$M_6 := (A + B) \times (H)$$

$$M_7 := D \times (G - E)$$

Then,

$$I := M_2 + M_3 - M_6 - M_7$$

$$J := M_4 + M_6$$

$$K := M_5 + M_7$$

$$L := M_1 - M_3 - M_4 - M_5$$

**Aim:** Huffman Coding

**Theory:** Huffman Coding is a technique of compressing data to reduce its size without losing any of the details. It was first developed by David Huffman.

Huffman Coding is generally useful to compress the data in which there are frequently occurring characters.

### **Time Complexity**

- The time complexity for encoding each unique character based on its frequency is  $O(n \log n)$ .
- Extracting minimum frequency from the priority queue takes place  $2 \cdot (n-1)$  times and its complexity is  $O(\log n)$ . Thus the overall complexity is  $O(n \log n)$ .

### **Algorithm**

HUFFMAN(C)

1:-n  $\leftarrow$  |C|

2:-Q  $\leftarrow$  C

3:-for i 1 to n - 1

4:-do allocate a new  
node z

5:-left[z]  $\leftarrow$  x  $\leftarrow$

6:-right[z]  $\leftarrow$  y  $\leftarrow$

7:-f[z]  $\leftarrow$  f[x] +

8:-INSERT(Q, z)

9:-return EXTRACT-  
MIN(Q)

**Aim: Activity Selection Problem**

**Theory:** Activity Selection is a problem in combinatorial optimization. The problem can be stated as follows: Given a set of activities with their start and finish times, select the maximum number of activities that can be performed by a single person, assuming that a person can only work on a single activity at a time. The activity selection problem can be solved using greedy algorithms, which make the best choice at each step and never look back. The key idea behind a greedy algorithm for the activity selection problem is to always select the activity that finishes earliest, as this will allow the maximum number of activities to be performed.

To implement a greedy algorithm for the activity selection problem, we first sort the activities in non-descending order of their finish times. After sorting, we start with the first activity and include it in the maximum-sized subset of activities. Then, we choose the next activity that starts after the finish time of the current activity and include it in the subset. This process is repeated until no more activities can be added to the subset. The algorithm can be implemented in a simple and straightforward manner, as follows:

Sort the activities in non-descending order of their finish times.

1. Initialize the current activity to be the first activity in the sorted array.
2. For each subsequent activity, if its start time is greater than or equal to the finish time of the current activity, include it in the subset and update the current activity to be this activity.
3. Repeat step 3 until all activities have been considered.
4. The time complexity of the greedy algorithm for the activity selection problem is  $O(n \log n)$ , where  $n$  is the number of activities. This is because sorting the activities takes  $O(n \log n)$  time and the rest of the algorithm takes  $O(n)$  time.

**Algorithm**

GREEDY-ACTIVITY-SELECTOR( $s, f$ )

```
1   $n \leftarrow \text{length}[s]$ 
2   $A \leftarrow \{a_1\}$ 
3   $i \leftarrow 1$ 
4  for  $m \leftarrow 2$  to  $n$ 
5      do if  $s_m \geq f_i$ 
6          then  $A \leftarrow A \cup \{a_m\}$ 
7               $i \leftarrow m$ 
8  return  $A$ 
```

**Time Complexity**

- The complexity of this problem is  $O(n \log n)$  when the list is not sorted.

When the sorted list is provided the complexity will be  $O(n)$ .

**AIM: To implement Longest Common Subsequence problem and analyze its time complexity.**

Theory:

The Longest Common Subsequence (LCS) problem in computer science involves finding the longest sequence of characters that appears in the same order in two given sequences (usually strings). It is a classic dynamic programming problem, where the goal is to identify the length and actual characters of this common subsequence. The dynamic programming approach efficiently solves this problem by building a table to store optimal subproblem solutions, leading to a time complexity of  $O(m * n)$ , where  $m$  and  $n$  are the lengths of the input sequences.

1. Algo: Create a table of dimension  $(n+1) * (m+1)$  where  $n$  and  $m$  are the lengths of  $x$  and  $y$  respectively. The first row and the first column are filled with zeros.
2. Fill each cell of the table using the following logic.
3. If the character corresponding to the current row and current column are matching, then fill the current cell by adding one to the diagonal element. Point an arrow to the diagonal cell.
4. Else take the maximum value from the previous column and previous row element for filling the current cell. Point an arrow to the cell with maximum value. If they are equal, point to any of them.
5. Step 2 is repeated until the table is filled.
6. The value in the last row and the last column is the length of the longest common subsequence.
7. In order to find the longest common subsequence, start from the last element and follow the direction of the arrow. The elements corresponding to  $()$  symbol form the longest common subsequence.

**Time complexity:** The time complexity of the Longest Common Subsequence (LCS) algorithm using dynamic programming is  $O(m * n)$ , where  $m$  and  $n$  are the lengths of the input sequences.

### Aim: **Optimal BST**

Theory: As we know that in binary search tree, the nodes in the left subtree have lesser value than the root node and the nodes in the right subtree have greater value than the root node.

We know the key values of each node in the tree, and we also know the frequencies of each node in terms of searching means how much time is required to search a node. The frequency and key-value determine the overall cost of searching a node. The cost of searching is a very important factor in various applications. The overall cost of searching a node should be less. The time required to search a node in BST is more than the balanced binary search tree as a balanced binary search tree contains a lesser number of levels than the BST. There is one way that can reduce the cost of a **binary search tree** is known as an **optimal binary search tree**.

Algorithm:

Input:

- An array `keys[]` of sorted distinct keys.
- An array `freq[]` of corresponding frequencies (probabilities) of each key.
- The number of keys `n`.

Output:

- The root of the optimal BST.

Initialization:

- Create a 2D table `cost` of size  $(n+1) \times (n+1)$  to store the cost of optimal BSTs for different subproblems.

Build the Cost Table:

- For each subarray length `len` from 1 to `n`:
  - For each starting index `i` from 0 to  $(n - len + 1)$ :
    - Calculate the cumulative frequency sum from `i` to `i + len - 1`.
    - For each possible root `r` in the subarray, calculate the cost:
      - $cost[i][i + len - 1] = \text{Sum}(\text{freq}[i \text{ to } i + len - 1]) + \text{Min}(cost[i][r-1] + cost[r+1][i + len - 1])$

Build the BST:

- Construct the optimal BST recursively using the information stored in the cost table:
  - The root of the optimal BST for keys `keys[i to j]` is the key `keys[r]` where `r` minimizes the cost in the `cost[i][j]` formula.
  - Recursively construct the left and right subtrees.

Return Root:

- Return the root of the optimal BST, which is stored in `cost[0][n-1]`.



Time complexity: The time complexity of constructing an Optimal Binary Search Tree (BST) using the dynamic programming approach is  $O(n^3)$ , where  $n$  is the number of keys.

#### AIM: Dijkstra algorithm

Theory: Dijkstra's algorithm is a popular algorithm in computer science and graph theory, named after its inventor Edsger W. Dijkstra. It is used to find the shortest paths between nodes in a weighted graph. The algorithm works for graphs with non-negative edge weights and is commonly employed in network routing protocols and various optimization problems.

Algorithm: 1. Set  $\text{dist}[\text{source}] = 0$ , and  $\text{dist}[v] = \text{infinity}$  for all other vertices.  
2. While there are vertices not in the set:  
a. Choose the vertex  $u$  with the smallest  $\text{dist}[u]$ .  
b. For each neighboring vertex  $v$  of  $u$ :  
- if  $(\text{dist}[u] + \text{weight}(u, v)) < \text{dist}[v]$ , update  $\text{dist}[v]$ .  
- Add  $u$  to the set of vertices with known distances.

Time complexity:

The time complexity of Dijkstra's algorithm depends on the implementation and the data structures used to efficiently select and update the minimum distance vertex. The common implementation using a priority queue or a binary heap achieves a time complexity of  $O((V + E) * \log(V))$ , where  $V$  is the number of vertices and  $E$  is the number of edges.

#### AIM: Bellman ford algorithm

Theory: The Bellman-Ford algorithm is a popular algorithm in graph theory used to find the shortest paths from a source vertex to all other vertices in a weighted graph. It can handle graphs with both positive and negative edge weights, as well as detect negative cycles. The algorithm is named after its inventors Richard Bellman and Lester Ford.

Algorithm: 1. Set  $\text{dist}[\text{source}] = 0$ , and  $\text{dist}[v] = \text{infinity}$  for all other vertices.  
2. Repeat  $|V| - 1$  times:  
a. For each edge  $(u, v)$  with weight  $w$ :  
- if  $(\text{dist}[u] + w) < \text{dist}[v]$ , update  $\text{dist}[v]$  and  $\text{prev}[v]$ .  
3. Check for negative cycles:  
- If there is any further relaxation in the distances, a negative cycle exists in the graph.

Time complexity: The time complexity of the Bellman-Ford algorithm is  $O(V * E)$ , where  $V$  is the number of vertices and  $E$  is the number of edges. The algorithm performs  $|V| - 1$  iterations, and in each iteration, it relaxes all edges in the graph. The worst-case time complexity arises when there are many edges to consider in each iteration.

### AIM: Minimum spanning tree

**Theory and algo:** A minimum spanning tree (MST) of a connected, undirected graph is a tree that spans all the vertices in the graph while minimizing the sum of edge weights. There are several algorithms to find the minimum spanning tree of a graph, and two of the most well-known ones are Prim's algorithm and Kruskal's algorithm.

### Prim's Algorithm:

Objective: Find the minimum spanning tree of a connected, undirected graph.

Steps:

Initialization:

- Start with an arbitrary vertex as the initial tree.
- Initialize a priority queue or a min-heap to keep track of candidate edges.

Grow the Tree:

- Repeat until all vertices are included in the tree:
  - Choose the edge with the minimum weight that connects a vertex in the tree to a vertex outside the tree.
  - Add the chosen edge and the new vertex to the tree.

Termination:

- When all vertices are included, the algorithm terminates, and the tree is a minimum spanning tree.

### Kruskal's Algorithm:

Objective: Find the minimum spanning tree of a connected, undirected graph.

Steps:

Sort Edges:

- Sort all the edges of the graph in non-decreasing order of their weights.

Initialization:

- Initialize an empty graph (the minimum spanning tree) and a disjoint-set data structure to keep track of connected components.

Add Edges:

- Iterate through the sorted edges:
  - Add the edge to the minimum spanning tree if it does not form a cycle in the current tree.
  - Use the disjoint-set data structure to check for cycles.

Termination:

- When the minimum spanning tree has  $|V| - 1$  edges (where  $|V|$  is the number of vertices), the algorithm terminates.

## Time Complexity:

- Prim's Algorithm:
  - With a simple implementation using an adjacency matrix, Prim's algorithm has a time complexity of  $O(V^2)$ , where  $V$  is the number of vertices.
  - With an optimized implementation using a priority queue or a min-heap, the time complexity is  $O((V + E) * \log(V))$ , where  $E$  is the number of edges.
- Kruskal's Algorithm:
  - With a straightforward implementation using a sorting algorithm, the time complexity is  $O(E * \log(E))$ , where  $E$  is the number of edges.
  - With efficient disjoint-set data structures, the time complexity is often dominated by the sorting step, making it  $O(E * \log(E))$ .

**AIM :** -To implement string matching algo and analyse the time complexity —

- a) Naive string matching
- b) Rabin karp
- c) Knuth Morris pratt

## Theory

### Algo of Navie string matching —

The naive string matching algorithm, also known as the Brute-Force or Simple Matching algorithm, is a straightforward method for finding occurrences of a pattern within a text. The algorithm compares the pattern with substrings of the text sequentially, character by character.

**Time Complexity:  $O(N^2)$**

**Auxiliary Space:  $O(1)$**

The algorithm for the same:

1. We will initialize an outer loop that will run from  $i = 0$  to  $i$  less than or equal to  $(n-m)$ . Since, after the index  $n-m$ , our searching space (or searching window) will exceed the length of the input string.
2. After the outer loop, we will initialize an inner loop that will run from  $j = 0$  to  $j$  less than  $m$  as this inner loop will range from 0 to the size of the input pattern.

3. Now, first we would check if the current input text character matches with the current input pattern character or not.
  - a) If a match is not found, we would break from the loop, and the inner loop will move one index far and start searching in the next window.
  - b) If a match is found, we would try to match the entire input pattern with the current window of the input text.

If we have traversed the entire searching window in the input text and we have found our pattern string, we would print the result.

Else, we would search in the next window.

### **KMP (Knuth morris pratt):**

**Time complexity :**  $O(n+m)$

The Knuth-Morris-Pratt (KMP) algorithm is a smarter string searcher compared to the basic approach. It uses a clever trick called the prefix function to avoid re-matching characters and significantly speed up the search. Imagine you're searching for a pattern in a text. KMP lets you jump ahead based on what you've already matched, making it faster and more efficient.

Think of it like this: if you're trying to find "ABABCABAB" in a text, KMP remembers that "ABA" appears twice within the pattern itself. So, if you find "AB" in the text but the next character doesn't match, instead of starting from scratch, you can jump back to the "ABA" part of the pattern and continue searching.

This smart jumping saves time and makes KMP a powerful tool for finding patterns in large amounts of data. So, if you need to find something quickly and efficiently, KMP is your friend!

Algo:-

### **Algorithm for the LLP Table**

- Take two variables i and j to trace the pattern.
- Initially,  $i \leftarrow 0$ , and  $j \leftarrow 2$ .
- Set  $LLP[0] = 0$ . The LLP value of the first character is always 0.
- if  $pattern[i] == pattern[j+1]$

$LLP[q] \leftarrow j+1$

- if  $pattern[i] \neq pattern[j+1]$

LLP [q] <- 0

### Step by Step Knuth Morris Pratt Algorithm

- Take two variables x and y to trace the index values of the string and pattern respectively.
- $x \leftarrow 1$ ,  $y \leftarrow 0$ ,  $len \leftarrow \text{length of the string}$ .
- While  $x < \text{string}[len]$

If  $\text{string}[x] == \text{pattern}[y + 1]$

It is a match.

$x \leftarrow x+1$  and  $y \leftarrow y+1$ .

- else If  $\text{string}[x] \neq \text{pattern}[y+1]$

It is a mismatch.

$x \leftarrow x+1$  and  $y \leftarrow \text{LLP}[y]$

### Robin karp —

Like the Naive Algorithm, the Rabin-Karp algorithm also check every substring. But unlike the Naive algorithm, the Rabin Karp algorithm matches the hash value of the pattern with the hash value of the current substring of text, and if the hash values match then only it starts matching individual characters. So Rabin Karp algorithm needs to calculate hash values for the following strings.

- Pattern itself
- All the substrings of the text of length m which is the size of pattern.

### Time complexity:

- The average and best-case running time of the Rabin-Karp algorithm is  $O(n+m)$ , but its worst-case time is  $O(nm)$ .
- The worst case of the Rabin-Karp algorithm occurs when all characters of pattern and text are the same as the hash values of all the substrings of  $T[]$  match with the hash value of  $P[]$ .

Algo:-

- First, determine the pattern's hash value.
- Iterate beginning at the start of the string:
- Find the hash value of the current m-character substring.
- If the current substring's hash value and the pattern's hash value are the same, see if the substring matches the pattern.
- Save the starting index as a valid response if they are identical. If not, move on to the following substrings.
- As the necessary response, give the starting indices.

**Aim: -Implement N Queen's problem using Back Tracking.**

Theory —

*The idea is to place queens one by one in different columns, starting from the leftmost column. When we place a queen in a column, we check for clashes with already placed queens. In the current column, if we find a row for which there is no clash, we mark this row and column as part of the solution. If we do not find such a row due to clashes, then we backtrack and return **false**.*

**Time Complexity:**  $O(N!)$

**Auxiliary Space:**  $O(N^2)$

Algo —

The algorithm is as follows -

- Define a character array board of dimensions
- $N \times N$  and initialize all its entries with '.', where '.' represents the empty cell.
- Start the process from the topmost row.
- If all queens are placed return true.
- Otherwise, for each of the columns in the current row do the following -
  - If it is possible to place the queen in the current cell
  - *i.e.* (row, col), place a queen in this cell.
  - Check if placing the queen in the current cell leads to a valid solution. If yes, then print the solution and terminate the program.
  - Otherwise, remove the queen from the current cell.
- If all the columns are tried for a given row, Return false to trigger the backtracking.

